

Great 👍 — let's finish **Week 4 Day 7 properly (portfolio-level)**

This is where your project becomes **interview-ready** 🚀

🔥 ✅ **Week 4 Day 7 — What you will do**

✅ Turn your code into a **professional project**

🧠 **STEP 1 — How YOU explain this project (Important)**

👉 In interview (60 sec):

I built a document-based AI tutor using a RAG system.

It uses ChromaDB for vector storage, Ollama embeddings, and a ReAct agent to dynamically select tools like search, calculation, and history.

The system enforces strict grounded responses using retrieved context and prevents hallucination using tool-based reasoning and guardrails.

✅ This is a **strong answer**

🔥 **STEP 2 — Architecture (VERY important)**

👉 Add this to README:

User Input

↓

ChatMemory

↓

ReAct Agent (LLM)

↓

Action Decision

↓

execute_action()

↓

search → VectorStore (ChromaDB)

↓

retrieve chunks

↓

Observation → back to LLM

↓

Final Answer

STEP 3 — DSA used in YOUR code

 This is what makes your project UNIQUE

 **Map to your code:**

DSA Concept	Where used
Queue	deque → chat_history
HashMap	OrderedDict → cache
Stack	self.stack (future use)
Graph	ReAct loop (state transitions)
DP	cache (memoization)
Array	embeddings vectors
Search	vector retrieval

STEP 4 — Key features (write this clearly)

 RAG (Retrieval-Augmented Generation)

- ✓ Vector search using embeddings
 - ✓ ReAct agent (LLM decides tools)
 - ✓ Tool execution system
 - ✓ Memory (chat history)
 - ✓ LRU Cache (fast repeated answers)
 - ✓ Observation cache (reduces cost)
 - ✓ Strict hallucination control
-

STEP 5 — Lessons Learned (IMPORTANT)

👉 Add this section:

1. Vector search alone is not enough — hybrid search is needed.
 2. Poor chunking destroys retrieval quality.
 3. LLM must be controlled using rules, not trusted blindly.
-

STEP 6 — Limitations (very important in interview)

Be honest:

- Retrieval accuracy depends on document quality
 - Vector search can return irrelevant chunks
 - ReAct loop can increase latency
-

STEP 7 — Week 5 plan (this is KEY)

Add this:

Next Improvements:

- ✓ Hybrid search (keyword + vector)
- ✓ Better chunking strategy

- ✓ FastAPI deployment
 - ✓ UI interface
 - ✓ Ranking + scoring system
-

✓ ✓ **FINAL README TEMPLATE (copy this)**

AI DSA Tutor with ReAct Agent

Overview

This project is an AI-powered tutor that answers DSA questions using a document-based retrieval system.

Architecture

[diagram above]

Features

- RAG system
- ReAct agent
- Tool execution
- Caching
- Memory

DSA Concepts Used

[list]

How it works

User → LLM → Tool → Retrieval → Answer

Limitations

[list]

Future Work

[list]

WHERE YOU ARE NOW

✅ You built:

- Vector DB ✅
- Retrieval ✅
- ReAct agent ✅
- Memory ✅
- Cache ✅

👉 This is already **advanced project**

FINAL TRUTH

✅ You are DONE with Week 4 ✅

🚀 You are ready for Week 5

NEXT STEP

Say:

👉 "Week 5 Day 1"

I'll teach you: ✅ **Hybrid Search (REAL upgrade — makes your system strong)**